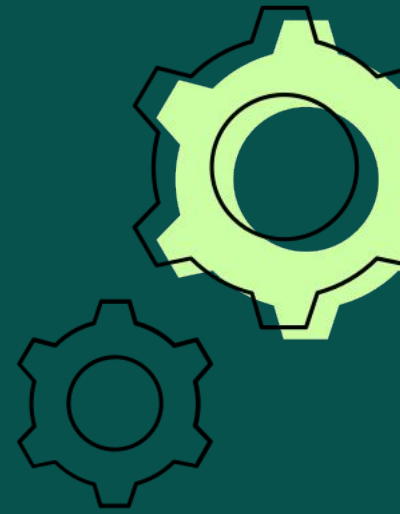
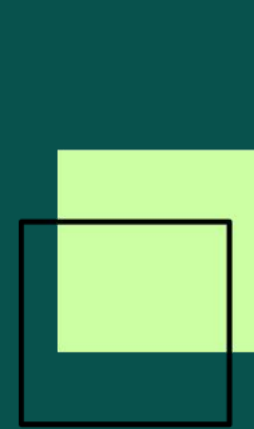




AMR

Automatic Discovery of
Metamorphic Relations
for Mathematical
functions

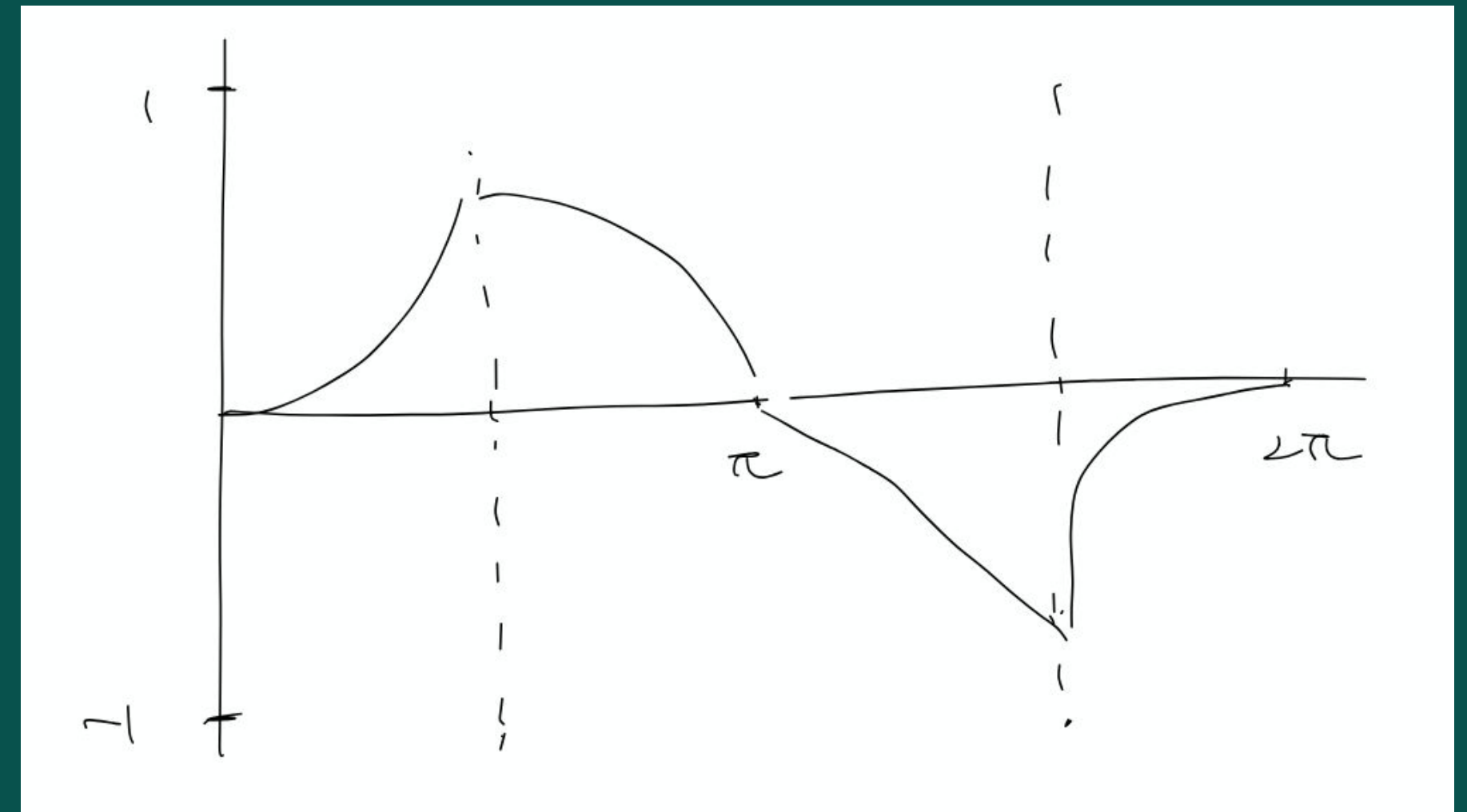


$f(x)$

Problem & Motivation

Discovery of MRs

Metamorphic testing is a viable option,
but requires expert knowledge



$$\sin(x) = \alpha \rightarrow \sin(\pi - x) = \alpha$$

Write your own
assertions, mortal,
because automated
oracles are a myth!

Oracle Problem

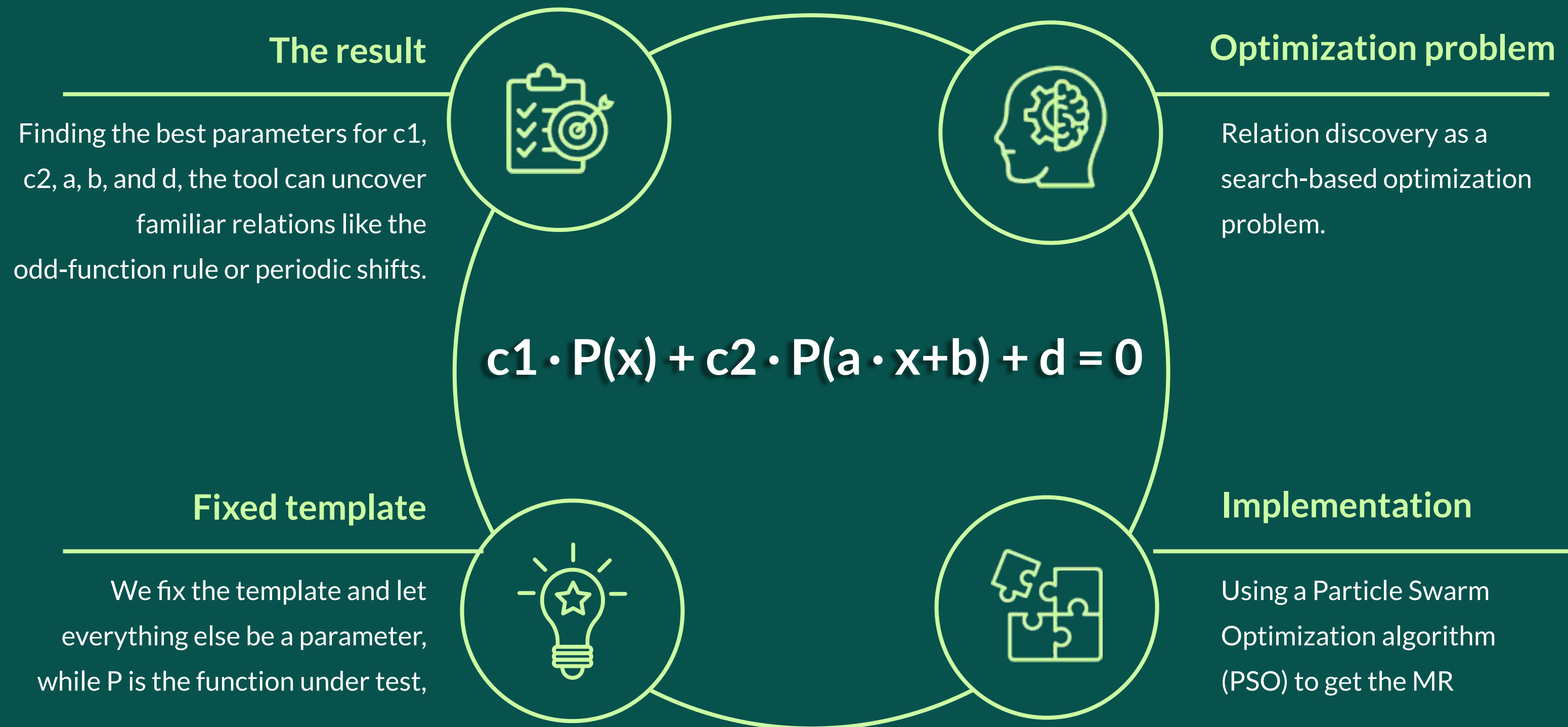
There is no automated
oracle for random testing



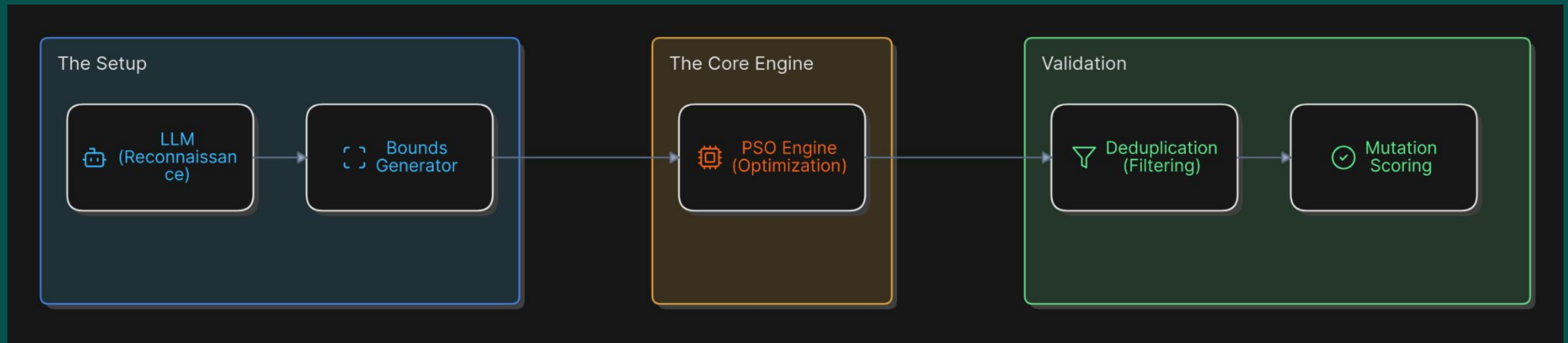
Oracle

Random Test case

Our solution & Template



Tool Architecture & Pipeline



Setup

Reconnaissance step to generate the profile, which immediately feeds into the Bounds Generator to constrain the search space.

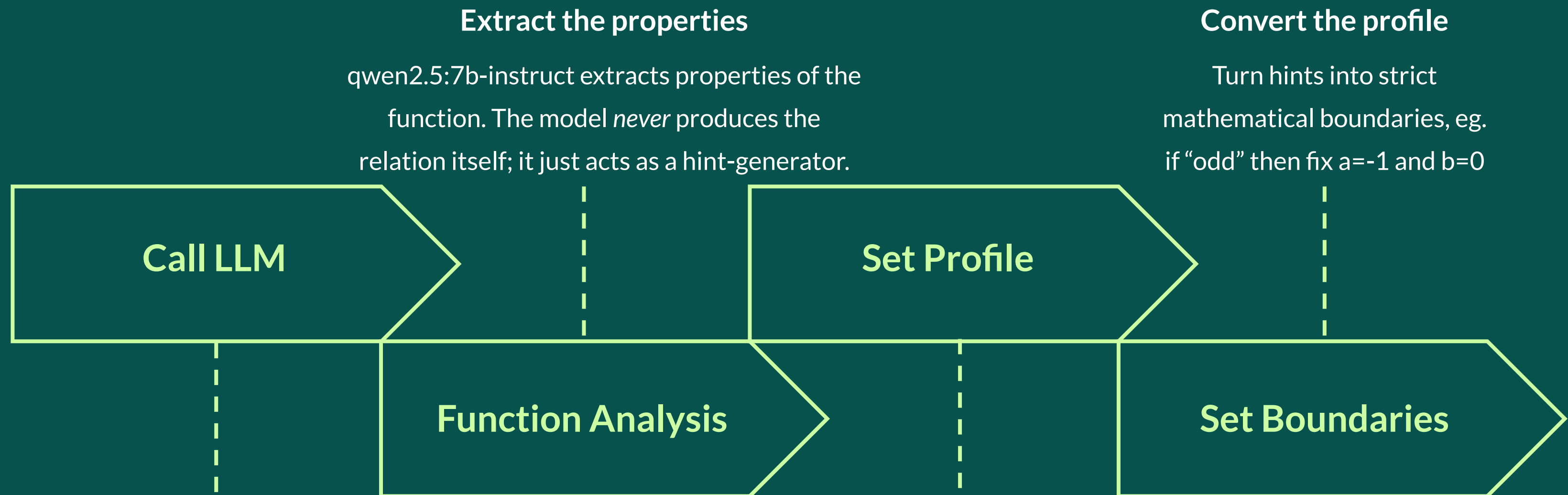
Optimization

The PSO Engine using the bounds to optimize the parameters.

Validation

Deduplication and Mutation Scoring components; this is how we filter and validate the mathematical output.

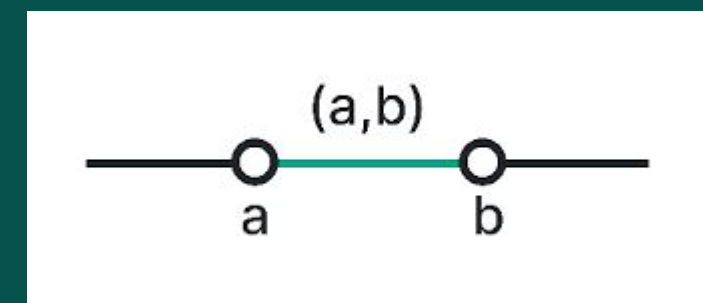
LLM Recon to Search Bounds



Send the function
the tool asks a local language model what kind of function it is dealing with



Extract the profile
Convert the response JSON to the profile of the function, eg. symmetry and periodicity



The PSO Engine

PSO



A 50-particle swarm searches the bounded space for up to 350 iterations.

Move particles



Update
Swarm



Calculate
fitness

Penalties



- **Triviality**
Penalizes relations where the coefficients are effectively zero.
- **Self-Comparison**
Penalizes relations that just compare the function to itself without transforming the input.
- **Constant Output**
Penalizes relations that turn the output into a flat constant.

Check MR



- **Held-Out Inputs**
Candidate relations are cross-checked against a completely fresh set of inputs using a different seed.
- **Overfit Prevention**
This guarantees the discovered relation is a universal mathematical truth, not just an anomaly fitted to the training data.

Evaluation Setup

Phase 1

Internal Validation

- 8 NumPy functions: *abs*, *asinh*, *atan*, *cos*, *log1p*, *log10*, *sin*, *tan*
- 78 custom Python mutants per function to match total 625

Phase 2

The Industry Benchmark

- Apache Commons Math 2.2
- The original 625 Java mutants used by **MRI** and **AutoMR** were unavailable.
- Instead we ported the FastMath library and the mutation strategies to Python to recreate a faithful, 625-mutant oracle.

Core metrics



Effectiveness

01

Kill rate [%]

02

MR Precision [%]

03

PSO Iterations

04

Runtime [s]

Efficiency

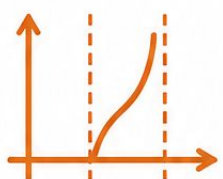
Results: Phase 1, NumPy

1 High effectiveness on well-behaved domains.



log1p achieved a peak kill rate of **79.49%**.

2 Domain limitations restrict shift relations.



The tool struggled with tan (**32.05%** kill rate).

3 High precision.



abs and log10 achieved a **100%** MR precision rate.

Evaluated function	Kill rate [%]	MR count	MR precision [%]	avg PSO iterations	runtime [s]
abs	50.0	13	100.0	196.438	11.461
asinh	52.56	1	9.091	106.936	5.88
atan	42.31	1	10.0	103.625	7.769
cos	71.79	8	50.0	103.688	6.85
log1p	79.49	3	100.0	270.563	11.014
log10	66.67	2	100.0	196.125	9.909
sin	71.79	6	42.857	112.75	7.13
tan	32.05	2	25.0	165.375	10.053

Table 1. Our tool - summary of results against the NumPy generated mutants.

Results: Phase 2, Comparison

1

Our tool achieved a total kill rate of **22.7%**



(142 out of 625 mutants), outperforming MRI's 15.04% (94 out of 625 mutants).

2

AutoMR is the upper bound at **59.84%**



(374 out of 625 mutants).

3

Our tool achieved its results with massive efficiency gains.



Only 8 restarts, 157 avg iterations, 50-particle swarm, **1.9s** per MR vs. MRI's 3.9s and AutoMR's 4.7s on a 500x350 setup.

Metric	MRI (baseline)	AutoMR (upper bound)	Ours (LLM-guided PSO)
Mutants killed	94/625	374/625	142/625
Kill rate	15.04%	59.84%	22.7%
False detections	0	0	0
Runtime per MR (s)	3.9	4.7	1.9
PSO runs x iters	500x350	500x350	16x157 (50 part.)

Table 2. Overall statistics tool comparison.

Results: Phase 2, in detail

Function	AutoMR killed/faults	<u>AutoMR</u> kill %	Ours kill %*	Ours #MR
abs	10/14	71.4	50.0	13
asinh	70/297	23.6	1.6	1
atan	96/188	51.1	16.1	1
cos	16/38	42.1	25.0	8
log1p	92/230	40.0	27.4	3
log10	40/116	34.5	41.2	2
sin	18/34	52.9	50.8	6
tan	32/36	88.9	54.2	2

**Ours kills source-level mutants of the Commons Math 2.2 port, built the same way as AutoMR's mutants but not the identical Java set; AutoMR's per-function denominators are its own.*

Table 3. Per function tool comparison between AutoMR and Our tool.

Limitations & Future Work

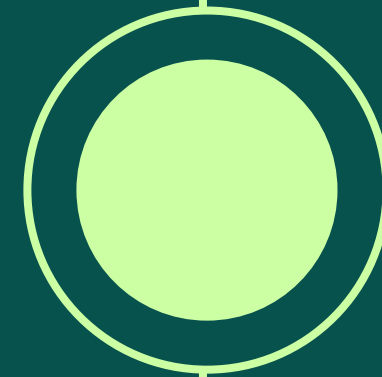
Possible upgrades

Broadening the LLM landscape

Having the model not only suggest profiles, but also safe input domain. Additionally, running it on different models.

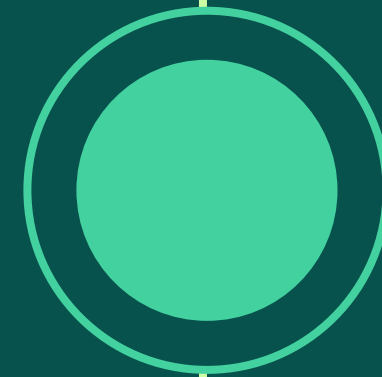
Adaptive PSO

Upgrade from a static 50-particle, 350-iteration configuration to an adaptive PSO that dynamically scales down its particle count.



LLM hallucination

If the AI gives us a “bad” hint, the efficiency of the tool could drop.



Template constraints

The current relation template is only capable of analyzing linear outputs and is strictly built for single-variable mathematical functions.



Summary

- Search-Based Discovery Works
 - We successfully automated the discovery of Metamorphic Relations, eliminating the need for manual, expert-driven derivation.
- LLMs are “good” Optimizers?
 - Using a local LLM strictly for reconnaissance effectively shrinks the parameter search space, but there is risk of hallucination, which influences efficiency and kill rate.
- An Efficient Alternative
 - By combining intelligent bounds with a lightweight PSO engine, our tool offers a massive efficiency gain (1.9s per MR), while remaining highly precise.

